

Préparation au Concours National Commun 2026 — Informatique MP

Épreuve d'Informatique

Filière MP — Durée : 4 heures

Arbres de Merkle et preuves d'inclusion : hachage, construction récursive, certificats logarithmiques

Auteur : Pr Agrégé El Hadiq Zouhair

cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit. **Pour aller plus loin.** Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet**, sur cpgeacademy.org.

Présentation et notations

Un *arbre de Merkle* permet de résumer une grande collection de données par un unique condensé, la *racine*, et de prouver qu'une donnée en fait partie à l'aide d'un *certificat* de taille logarithmique. C'est un outil central des blockchains (preuves d'inclusion, listes d'autorisation, stockage vérifiable). Dans tout le problème, les *feuilles* sont des entiers, et l'on combine deux nœuds par la fonction

$$\text{parent}(a, b) = (31a + b) \bmod p, \quad p = 1000003 \text{ (premier).}$$

Conventions. Le langage est Python 3. Pour une liste L , $\text{len}(L)$ est sa longueur, $L[i]$ son terme d'indice i (≥ 0), $L[-1]$ son dernier terme. $\lceil x \rceil$ est la partie entière supérieure, $\log_2$ le logarithme binaire. Les questions sont numérotées **en continu**.

ÉNONCÉ

Partie I — La fonction de combinaison

Question 1. Écrire $\text{parent}(a, b)$ conforme à la définition ci-dessus.

Question 2. Calculer à la main $\text{parent}(1, 2)$, $\text{parent}(3, 4)$ et $\text{parent}(33, 97)$.

Question 3. On dit que parent est *déterministe*. Expliquer pourquoi ce caractère déterministe est indispensable pour que deux nœuds différents du réseau, partant des mêmes feuilles, calculent la même racine.

Question 4. On admet, pour la suite, que parent est *résistante aux collisions* : on ne sait pas exhiber deux couples distincts $(a, b) \neq (a', b')$ tels que $\text{parent}(a, b) = \text{parent}(a', b')$. Rappeler pourquoi une fonction $h : \mathbb{N}^2 \rightarrow \{0, \dots, p-1\}$ ne peut pas être *injective*, et donc pourquoi la résistance aux collisions est une hypothèse *calculatoire* (« difficile à violer ») et non une impossibilité mathématique.

Partie II — Construction de la racine

La racine se calcule par niveaux : tant qu'il reste plus d'un nœud, si le nombre de nœuds est impair on **duplique le dernier**, puis on remplace la liste par celle des **parent** de couples consécutifs.

Question 5. Écrire `racine_merkle(feuilles)` (liste non vide) renvoyant la racine.

Question 6. Dérouler le calcul de `racine_merkle([1, 2, 3, 4])` niveau par niveau et donner la valeur de la racine.

Question 7. Faire de même pour `racine_merkle([1, 2, 3])` en explicitant l'étape de duplication.

Question 8. Soit n le nombre de feuilles. En notant $t_0 = n$ et $t_{k+1} = \lceil t_k/2 \rceil$ la taille du niveau $k+1$, montrer que le nombre de niveaux est $O(\log n)$ et que le nombre total d'appels à **parent** est un $O(n)$.

Partie III — Preuves d'inclusion

Question 9. Écrire `preuve_inclusion(feuilles, i)` renvoyant la liste des couples (frère, côté), avec côté valant **'L'** ou **'R'**, rencontrés en remontant de la feuille d'indice i jusqu'à la racine.

Question 10. Écrire `verifier(feuille, preuve, racine)` qui recalcule la racine depuis la feuille et la preuve et renvoie un booléen. Vérifier sur l'exemple $[1, 2, 3, 4]$, $i = 0$.

Question 11. Montrer que la taille d'une preuve d'inclusion est le nombre de niveaux, soit $O(\log n)$, et commenter l'intérêt (comparer au coût de transmettre toutes les feuilles).

Question 12. Soundness. Démontrer par récurrence sur la hauteur h de l'arbre que, sous l'hypothèse de résistance aux collisions, si `verifier(feuille, preuve, racine)` renvoie **True** alors `feuille` figure effectivement parmi les feuilles ayant produit `racine`.

Partie IV — Hauteur et mise à jour

Question 13. Montrer que la hauteur de l'arbre (nombre de niveaux au-dessus des feuilles) vaut $H(n) = \lceil \log_2 n \rceil$ pour $n \geq 1$ (avec $H(1) = 0$). Donner $H(n)$ pour $n \in \{1, 2, 3, 4, 5, 8, 9\}$.

Question 14. Lorsqu'une seule feuille change de valeur, quels nœuds de l'arbre doivent être recalculés ? En déduire qu'une mise à jour de la racine coûte $O(\log n)$ appels à **parent** (et non $O(n)$). Écrire `maj_racine(feuilles, i, v)` qui renvoie la nouvelle racine lorsque la feuille d'indice i prend la valeur v , en **réutilisant** `racine_merkle`. *On ne demande pas ici l'implémentation optimale en $O(\log n)$, mais on justifiera la borne.*

CORRIGÉ

Partie I — La fonction de combinaison

Question 1.

```
1 def parent(a, b):
2     return (31 * a + b) % 1000003
```

Question 2. $\text{parent}(1, 2) = 31 \cdot 1 + 2 = 33$; $\text{parent}(3, 4) = 31 \cdot 3 + 4 = 97$; $\text{parent}(33, 97) = 31 \cdot 33 + 97 = 1023 + 97 = 1120$.

Question 3. Le calcul de la racine doit donner *le même* résultat sur tous les nœuds du réseau pour que le consensus soit possible. Si `parent` pouvait renvoyer des valeurs différentes pour les mêmes entrées (aléa, dépendance à l'environnement), deux nœuds honnêtes partant des mêmes feuilles obtiendraient des racines différentes et ne pourraient pas s'accorder. Le déterminisme garantit la *reproductibilité*.

Question 4. L'ensemble de départ $\mathbb{N}^2$ est infini (ou, en pratique, de cardinal bien plus grand que p), tandis que l'ensemble d'arrivée $\{0, \dots, p-1\}$ est de cardinal fini p . Par le principe des tiroirs, h ne peut être injective : des collisions *existent* nécessairement. La résistance aux collisions n'affirme donc pas leur absence, mais qu'il est *calculatoirement difficile* d'en exhiber une : c'est une hypothèse de sécurité, pas un théorème d'injectivité. $\square$

Partie II — Construction de la racine

Question 5.

```
1 def racine_merkle(feuilles):
2     niveau = list(feuilles)
3     while len(niveau) > 1:
4         if len(niveau) % 2 == 1:
5             niveau.append(niveau[-1]) # duplique le dernier
6             niveau = [parent(niveau[i], niveau[i+1])
7                       for i in range(0, len(niveau), 2)]
8     return niveau[0]
```

Question 6. Niveau 0 = $[1, 2, 3, 4]$; $\text{parent}(1, 2) = 33$, $\text{parent}(3, 4) = 97$, donc niveau 1 = $[33, 97]$; $\text{parent}(33, 97) = 1120$. Racine = **1120**.

Question 7. Niveau 0 = $[1, 2, 3]$, impair $\Rightarrow$ on duplique : $[1, 2, 3, 3]$. $\text{parent}(1, 2) = 33$, $\text{parent}(3, 3) = 31 \cdot 3 + 3 = 96$, donc niveau 1 = $[33, 96]$; $\text{parent}(33, 96) = 1023 + 96 = 1119$. Racine = **1119**.

Question 8. On a $t_{k+1} = \lceil t_k/2 \rceil \leq t_k/2 + 1$. Par récurrence $t_k \leq n/2^k + 2$, donc $t_k \leq 2$ dès que $n/2^k \leq 1$, c'est-à-dire $k \geq \log_2 n$: il y a $O(\log n)$ niveaux. Le nombre d'appels à `parent` au niveau k est $\lceil t_k/2 \rceil$, et le total vaut $\sum_{k \geq 1} t_k \leq \sum_{k \geq 1} (n/2^k + 2) \leq n + O(\log n) = O(n)$. $\square$

Partie III — Preuves d'inclusion

Question 9.

```

1 def preuve_inclusion(feuilles, i):
2     niveau = list(feuilles)
3     preuve = []
4     idx = i
5     while len(niveau) > 1:
6         if len(niveau) % 2 == 1:
7             niveau.append(niveau[-1])
8         if idx % 2 == 0:
9             preuve.append((niveau[idx+1], 'R')) # frere a droite
10        else:
11            preuve.append((niveau[idx-1], 'L')) # frere a gauche
12        niveau = [parent(niveau[j], niveau[j+1])
13                  for j in range(0, len(niveau), 2)]
14        idx //= 2
15    return preuve

```

Question 10.

```

1 def verifier(feuille, preuve, racine):
2     acc = feuille
3     for frere, cote in preuve:
4         if cote == 'R':
5             acc = parent(acc, frere)
6         else:
7             acc = parent(frere, acc)
8     return acc == racine

```

Exemple : `preuve_inclusion([1, 2, 3, 4], 0) = [(2, 'R'), (97, 'R')]`. La vérification calcule `parent(1, 2) = 33` puis `parent(33, 97) = 1120`, égal à la racine : `True`.

Question 11. La preuve contient exactement un couple par niveau au-dessus de la feuille, soit $H(n) = \lceil \log_2 n \rceil = O(\log n)$ couples. On transmet donc $O(\log n)$ valeurs au lieu des n feuilles : pour $n = 10^6$, une vingtaine de valeurs suffisent. C'est ce qui rend les preuves de Merkle utilisables sur une blockchain, où l'espace de stockage on-chain est coûteux.

Question 12. Récurrence sur h . Base $h = 0$: l'arbre est réduit à une feuille, la racine est cette feuille, et la preuve est vide ; `verifier` renvoie `True` seulement si `feuille = racine`, donc la feuille est bien celle ayant produit la racine. *Hérédité* : supposons le résultat vrai pour tout arbre de hauteur h , et soit un arbre de hauteur $h + 1$. La vérification calcule d'abord $r' = \text{parent}(\text{feuille}, f)$ (ou `parent(f, feuille)`), puis vérifie que r' , muni du reste de la preuve, mène à la racine : c'est une preuve d'inclusion valide de r' dans l'arbre des nœuds de niveau 1, de hauteur h . Par hypothèse de récurrence, r' est bien un nœud de ce niveau, produit par un couple réel de l'arbre. Comme `parent` résiste aux collisions, le seul antécédent connu de r' est le couple effectivement présent : c'est `(feuille, f)`. Donc `feuille` figure parmi les feuilles ayant produit la racine. $\square$

Partie IV — Hauteur et mise à jour

Question 13. À chaque niveau, la taille passe de t à $\lceil t/2 \rceil$; partant de n , il faut le plus petit H tel que $\lceil \dots \lceil n/2 \rceil \dots \rceil = 1$ après H divisions, soit $2^{H-1} < n \leq 2^H$ pour $n \geq 2$, d'où $H(n) = \lceil \log_2 n \rceil$ (et $H(1) = 0$). On obtient

$$H(1) = 0, H(2) = 1, H(3) = 2, H(4) = 2, H(5) = 3, H(8) = 3, H(9) = 4.$$

Question 14. Seuls les nœuds situés *sur le chemin* de la feuille modifiée jusqu'à la racine changent de valeur : à chaque niveau, un seul nœud dépend de la feuille modifiée (son ancêtre). Il y a

$H(n) = O(\log n)$ tels nœuds ; recalculer chacun coûte un appel à `parent`, d'où une mise à jour en $O(\log n)$ appels. Une implémentation simple, correcte mais non optimale, recalcule toute la racine :

```
1 def maj_racine(feuilles, i, v):  
2     nouvelles = list(feuilles)  
3     nouvelles[i] = v  
4     return racine_merkle(nouvelles)
```

Cette version coûte $O(n)$; l'argument ci-dessus montre qu'une version conservant les nœuds inchangés atteindrait $O(\log n)$. $\square$

Vérifications numériques (exécutées en Python) : `racine_merkle([1, 2, 3, 4]) = 1120`, `racine_merkle([1, 2, 3]) = 1119`, et les hauteurs annoncées correspondent à $\lceil \log_2 n \rceil$.